[1부] 머신러닝
3장 분류

핸즈온 머신러닝(3판)

ㅇㅇ대학교 ㅇㅇ학과
홍길동

시작하기 전에

지은이: 오렐리앙 제롱 Aurélien Géron

머신러닝 컨설턴트. 2013년에서 2016년까지 구글에서 유튜브 동영상 분류 팀을 이끌었다. 2002년에서 2012년까지 프랑스의 모바일 ISP 선두주자인 위퍼스트(Wifirst)를 설립하고 CTO로 일했다. 2001년에는 폴리콘셀(Polyconseil)을 설립하고 CTO로 일했다. 이 회사는 지금 전기차 공유 서비스인 오토립(Autolib')을 운영하고 있다.
그전에는 재무(J. P. 모건과 소시에테 제네랄(Société Générale)), 방위(캐나다 국방부), 의료(수혈) 등 다양한 분야에서 엔지니어로 일했고, C++, WiFi, 인터넷 구조에 관한 몇 권의 기술 서적을 썼으며 한 프랑스 공과대학에서 컴퓨터과학을 가르쳤다.

옮긴이: 박해선 haesun.park@tensorflow.blog

기계공학을 전공했지만 졸업 후엔 줄곧 코드를 읽고 쓰는 일을 했다. 블로그(*tensorflow.blog*)에 글을 쓰고 머신러닝과 딥러닝에 관한 책을 집필, 번역하면서 소프트웨어와 과학의 경계를 흥미롭게 탐험하고 있다.
『챗GPT로 대화하는 기술』(한빛미디어, 2023), 『혼자 공부하는 머신러닝+딥러닝』(한빛미디어, 2020), 『혼자 공부하는 데이터 분석 with 파이썬』(한빛미디어, 2023), 『Do it! 딥러닝 입문』(이지스퍼블리싱, 2019) 등 집필했다.

[코드 예제] *https://bit.ly/homl3-git* 에서 주피터 노트북으로 제공

3판의 주요 변경 내용

1. 최신 라이브러리 버전으로 전체 코드 업데이트
2. 특성 이름 추적, 히스토그램 기반 그레이디언트 부스팅, 레이블 전파 등 사이킷런에 새롭게 추가된 다양한 기능
3. 하이퍼파라미터 튜닝을 위한 케라스 튜너(Keras Tuner) 라이브러리, 자연어 처리를 위한 허깅 페이스(Hugging Face)의 트랜스포머스(Transformers) 라이브러리 및 케라스의 새로운 전처리 및 데이터 증식 층
4. 여러 비전 모델(ResNeXt, DenseNet, MobileNet, CSPNet, EfficientNet)과 올바른 모델을 선택하기 위한 가이드라인
5. <15장 RNN과 CNN을 사용한 시퀀스 처리>: 합성된 시계열 대신 시카고 버스 및 철도 탑승객 데이터를 분석하며 ARMA 모델과 그 변형
6. <16장 RNN과 어텐션을 사용한 자연어 처리>: 인코더-디코더 RNN, 트랜스포머 모델을 사용한 영어-스페인어 번역 모델 구축, 스위치 트랜스포머(Switch Transformer), DistilBERT, T5, PaLM(사고 사슬 프롬프트 포함)과 같은 언어 모델 비전 트랜스포머(ViT) 소개, DeiT, 퍼시비어, DINO와 같은 트랜스포머 기반 비전 모델을 비롯한 CLIP, DALL·E, 플라밍고, GATO 등 몇 가지 대형 멀티모달 모델 개요
7. <17장 오토인코더, GAN 그리고 확산 모델>: 확산 모델 소개, DDPM 구현
8. <19장 대규모 텐서플로 모델 훈련과 배포>: 구글 클라우드 AI 플랫폼에서 구글 버텍스 AI로 마이그레이션, 대규모 하이퍼파라미터 검색을 위한 분산 케라스 튜너, TensorFlow.js 코드
9. PipeDream과 Pathways를 비롯한 추가적인 분산 훈련 기법

[참조] https://homl.info/changes3

이 책의 학습 목표

1부 머신러닝

1장: 한눈에 보는 머신러닝	데이터 과학자가 꼭 알아야 할 기초 개념과 용어
2장: 머신러닝 프로젝트 처음부터 끝까지	주택 가격을 예측하는 회귀 작업을 살펴보면서 선형 회귀, 결정 트리, 랜덤 포레스트 등 여러 알고리즘 학습
3장: 분류	분류 시스템 학습
4장: 모델 훈련	신경망 구축에 필요한 모델 훈련 알고리즘 학습
5장: 서포트 벡터 머신	SVM의 핵심 개념, 사용 방법, 작동 원리 학습
6장: 결정 트리	결정 트리의 훈련, 시각화, 예측 방법과 사이킷런의 CART 훈련 알고리즘 등
7장: 앙상블 학습과 랜덤 포레스트	투표 기반 분류기, 배깅과 페이스팅 앙상블, 랜덤 포레스트, 부스팅, 스태킹 앙상블 등
8장: 차원 축소	고차원 공간과 차원 축소 기법
9장: 비지도 학습	비지도 학습과 알고리즘

이 책의 학습 목표

2부 신경망과 딥러닝

10장: 케라스를 사용한 인공 신경망 소개	인공 신경망과 케라스를 이용한 구현 방법
11장: 심층 신경망 훈련	심층 신경망의 문제와 해결 방법
12장: 텐서플로를 사용한 사용자 정의 모델과 훈련	텐서플로와 저수준 파이썬 API를 이용한 사용자 정의 모델과 훈련 알고리즘
13장: 텐서플로를 사용한 데이터 적재와 전처리	tf.data API와 TFRecord 포맷, 케라스 전처리 층 및 tf.data API 사용 방법
14장: 합성곱 신경망을 사용한 컴퓨터 비전	CNN 구성 요소, 케라스를 사용한 구현 방법
15장: RNN과 CNN을 사용한 시퀀스 처리	RNN 개념과 WaveNet 구현
16장: RNN과 어텐션을 사용한 자연어 처리	문장 수준의 RNN과 어텐션 메커니즘
17장: 오토인코더, GAN 그리고 확산 모델	오토인코더 차원 축소, 특성 추출, 비지도 사전 훈련 방법과 생성 모델
18장: 강화 학습	강화 학습 개념, 정책 그레이디언트, 심층 Q-네트워크
19장: 대규모 텐서플로 모델 훈련과 배포	TF 서빙과 구글 버텍스 AI 플랫폼에 모델을 배포하는 방법

이 책의 학습 목표

3부 부록

부록 A: 연습문제 정답

부록 B: 머신러닝 프로젝트 체크리스트

부록 C: 자동 미분

부록 D: 특수한 데이터 구조

부록 E: 텐서플로 그래프

Contents

- 3장: 분류

3.1 MNIST
3.2 이진 분류기 훈련
3.3 성능 측정
3.4 다중 분류
3.5 오류 분석
3.6 다중 레이블 분류
3.7 다중 출력 분류

3장 분류

분류 시스템 학습

3.1 MNIST(1)

- MNIST 데이터셋
  - 고등학생과 미국 인구조사국 직원들이 손으로 쓴 70,000개의 작은 숫자 이미지. 각 이미지에는 어떤 숫자를 나타내는지 레이블 되어 있음(학습용으로 매우 많이 사용됨)
  - MNIST 데이터셋 내려받기

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```

  - sklearn.datasets 패키지의 함수
    - 실전 데이터셋을 다운로드하기 위한 fetch_* 함수
    - 사이킷런에 번들로 포함된 소규모 데이터셋을 로드하기 위한 load_* 함수
    - 테스트에 유용한 가짜 데이터셋을 생성하기 위한 make_* 함수
  - 사이킷런에서 읽어 들인 데이터셋들의 일반적인 딕셔너리 구조
    - DESCR: 데이터셋 설명
    - data: 입력 데이터, 일반적으로 2D 넘파이 배열
    - target: 레이블, 일반적으로 1D 넘파이 배열

3.1 MNIST(2)

- fetch_openml() 함수는 기본적으로 입력을 판다스 데이터프레임, 레이블을 판다스 시리즈로 반환
  - MNIST 데이터셋은 이미지이므로 데이터프레임이 잘 맞지 않음
  - 따라서 as_frame=False로 지정하여 넘파이 배열로 데이터를 받음

```
>>> X, y = mnist.data, mnist.target
>>> X
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
>>> X.shape
(70000, 784)
>>> y
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
>>> y.shape
(70000,)
```

3.1 MNIST(3)

- 데이터셋에서 이미지 하나를 확인(그림 3-1)
  - 샘플의 특성 벡터를 추출해서 28×28 배열로 크기를 바꾸고 맷플롯립의 imshow() 함수를 사용
  - cmap="binary"로 지정해 0을 흰색, 255를 검은색으로 나타내는 흑백(grayscale) 컬러 맵을 사용

```
import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
plt.show()
```

그림 3-1 MNIST 이미지

  - 숫자 5로 보이는 그림의 실제 레이블을 확인

```
>>> y[0]
'5'
```

3.1 MNIST(4)

- MNIST 이미지 샘플

그림 3-2 MNIST 데이터셋에서 추출한 숫자 이미지

3.1 MNIST(5)

- fetch_openml()이 반환한 MNIST 데이터셋은 이미 훈련 세트(앞쪽 60,000개 이미지)와 테스트 세트(뒤쪽 10,000개 이미지)로 분리

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

- 훈련 세트는 이미 섞여 있어서 모든 교차 검증 폴드를 비슷하게 만듦
  (하나의 폴드라도 특정 숫자가 누락되면 안 됨)
- 어떤 학습 알고리즘은 훈련 샘플의 순서에 민감해서 많은 비슷한 샘플이 연이어 나타나면 성능이 나빠짐
  - 데이터셋을 섞어서 이런 문제를 방지

3.2 이진 분류기 훈련

- '5-감지기'는 '5'와 '5 아님' 두 개의 클래스를 구분할 수 있는 이진 분류기(binary classifie)
  - 타깃 벡터 생성

```
y_train_5 = (y_train == '5')   # 5는 True고, 다른 숫자는 모두 False
y_test_5 = (y_test == '5')
```

- SGDClassifier 모델을 만들고 전체 훈련 세트를 사용해 훈련
  - 확률적 경사 하강법(Stochastic Gradient Descent, SGD)

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

- 숫자 5의 이미지를 감지

```
>>> sgd_clf.predict([some_digit])
array([ True])
```

3.3 성능 측정(1)

3.3.1 교차 검증을 사용한 정확도 측정

- cross_val_score() 함수
  - 폴드가 3개인 k-폴드 교차 검증을 사용해 SGDClassifier 모델을 평가

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.95035, 0.96035, 0.9604])
```

- 모든 이미지를 가장 많이 등장하는 클래스 (여기서는 음성 클래스, 즉 '5 아님')로 분류하는 더미 분류기를 만들어 비교

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
# False가 출력됩니다. 즉, True로 예측된 것이 없습니다.
print(any(dummy_clf.predict(X_train)))
```

- 모델의 정확도 확인

```
>>> cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.90965, 0.90965, 0.90965])
```

3.3 성능 측정(2)

교차 검증 구현

- 사이킷런이 제공하는 기능보다 교차 검증 과정을 더 많이 제어할 필요가 있을 경우
  - 교차 검증 기능을 직접 구현
- 다음 코드는 사이킷런의 cross_val_score() 함수와 거의 같은 작업을 수행하고 동일한 결과를 출력

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3) # 데이터셋이 미리 섞여 있지 않다면
                                      # shuffle=True를 추가하세요.

for train_index, test_index in skfolds.split(X_train, y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]
    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
    print(n_correct / len(y_pred))  # 출력: 0.95035, 0.96035, 0.9604
```

3.3 성능 측정(3)

3.3.2 오차 행렬

- 오차 행렬 - 모든 A/B 쌍에 대해 클래스 A의 샘플이 클래스 B로 분류된 횟수를 세는 것
- cross_val_predict() 함수
  - 실제 타깃과 비교할 수 있도록 예측값 만들기
  - k-폴드 교차 검증을 수행하지만 평가 점수를 반환하지 않고 각 테스트 폴드에서 얻은 예측을 반환

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

- confusion_matrix() 함수
  - 오차 행렬을 만들기

```
>>> from sklearn.metrics import confusion_matrix
>>> cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
array([[53892,  687],
       [ 1891, 3530]])
```

3.3 성능 측정(4)

- 오차 행렬의 행은 실제 클래스, 열은 예측한 클래스를 나타냄
  - 예제 행렬의 첫 번째 행은 '5 아님' 이미지(음성 클래스 negative class)에 대한 것
    - 53,892개를 '5 아님'으로 정확하게 분류 - 진짜 음성(true negative)
    - 나머지 687개는 '5'라고 잘못 분류 - 거짓 양성(false positive) 또는 1종 오류(type I error)
  - 두 번째 행은 '5' 이미지(양성 클래스 positive class)에 대한 것
    - 1,891개를 '5 아님'으로 잘못 분류 - 거짓 음성(false negative) 또는 2종 오류(type II error)
    - 나머지 3,530개를 정확히 '5'라고 분류 - 진짜 양성(true positive)
- 완벽한 분류기라면 진짜 양성과 진짜 음성만 가지고 있을 것이므로 오차 행렬의 주대각선(왼쪽 위에서 오른쪽 아래로)만 0이 아닌 값이 됨

```
>>> y_train_perfect_predictions = y_train_5   # 완벽한 분류기일 경우
>>> confusion_matrix(y_train_5, y_train_perfect_predictions)
array([[54579,     0],
       [    0,  5421]])
```

3.3 성능 측정(5)

- 분류기의 정밀도(precision) - 양성 예측의 정확도
  - TP는 진짜 양성의 수이고, FP는 거짓 양성의 수
- 정밀도는 재현율(recall)이라는 또 다른 지표와 같이 사용하는 것이 일반적
  - 재현율은 분류기가 정확하게 감지한 양성 샘플의 비율로, 민감도(sensitivity) 또는 진짜 양성 비율(true positive rate ,TPR)
  - FN은 거짓 음성의 수

식 3-1 정밀도 $$정밀도 = \frac{TP}{TP + FP}$$

식 3-2 재현율 $$재현율 = \frac{TP}{TP + FN}$$

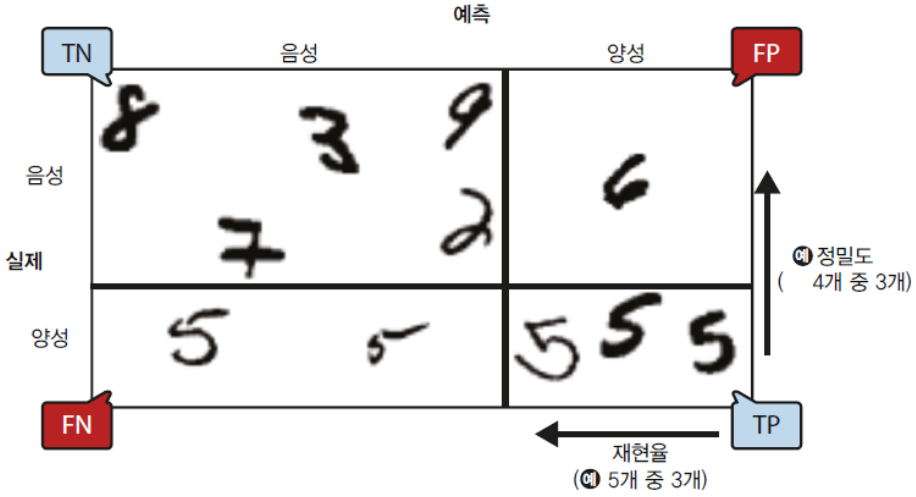

그림 3-3 오차 행렬 그림 - 진짜 음성 샘플(TN), 거짓 양성(FP), 거짓 음성(FN), 진짜 양성(TP)

3.3 성능 측정(6)

3.3.3 정밀도와 재현율

- 사이킷런은 정밀도와 재현율을 포함하여 분류기의 지표를 계산하는 여러 함수를 제공

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
0.8370879772350012
>>> recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
0.6511713705958311
```

3.3 성능 측정(7)

- F_1 점수(F_1 score)는 정밀도와 재현율의 조화 평균(harmonic mean)

식 3-3 F_1 점수

$$F_1 = \frac{2}{\frac{1}{\text{정밀도}} + \frac{1}{\text{재현율}}} = 2 \times \frac{\text{정밀도} \times \text{재현율}}{\text{정밀도} + \text{재현율}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

- f1_score() 함수 - F_1 점수를 계산

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```

3.3 성능 측정(8)

3.3.4 정밀도/재현율 트레이드오프

- SGDClassifier의 분류 결정 과정
  - 결정 함수(decision function)를 사용하여 각 샘플의 점수를 계산
  - 점수가 임곗값보다 크면 샘플을 양성 클래스에 할당하고 그렇지 않으면 음성 클래스에 할당
  - 결정 임곗값이 높이면 재현율이 낮아지고 정밀도가 올라감
  - 임곗값을 내리면 재현율이 높아지고 정밀도가 줄어듬

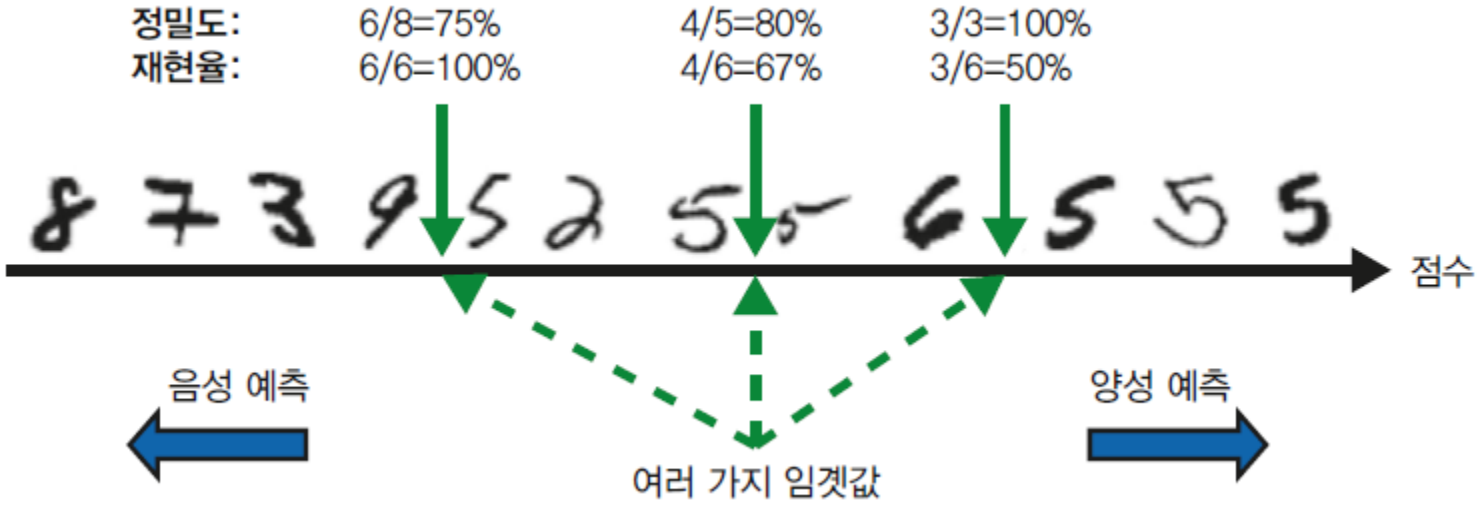

그림 3-4 이 정밀도/재현율 트레이드오프 이미지는 분류기가 만든 점수 순으로 나열. 선택한 결정 임곗값 위에 있는 이미지를 양성으로 판단. 임곗값이 높을수록 재현율은 낮아지고 반대로 (보통) 정밀도는 높아짐

3.3 성능 측정(9)

- decision_function() 메서드
  - 각 샘플의 점수를 확인하고 이 점수를 기반으로 원하는 임곗값을 정해 예측 만들기
  - SGDClassifier의 임곗값이 0이므로 predict() 메서드와 같은 결과(즉, True)를 반환

```
>>> y_scores = sgd_clf.decision_function([some_digit])
>>> y_scores
array([2164.22030239])
>>> threshold = 0
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([ True])
```

  - 임곗값을 높여서 재현율이 줄어드는 것을 확인

```
>>> threshold = 3000
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([False])
```

3.3 성능 측정(10)

- 적절한 임곗값 결정
  - 먼저 cross_val_predict() 함수를 사용해 훈련 세트에 있는 모든 샘플의 점수를 계산
    - 이번에는 예측 결과가 아니라 결정 점수를 반환하도록 지정

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             method="decision_function")
```

  - 이 점수로 precision_recall_curve() 함수를 사용하여 가능한 모든 임곗값에 대해 정밀도와 재현율을 계산

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

3.3 성능 측정(11)

- 맷플롯립을 이용해 임곗값의 함수로 정밀도와 재현율 시각화

```
plt.plot(thresholds, precisions[:-1], "b--", label="정밀도", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="재현율", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="임곗값")
[...] # 그리드, 범례, 축, 레이블, 동그라미를 추가합니다.
plt.show()
```

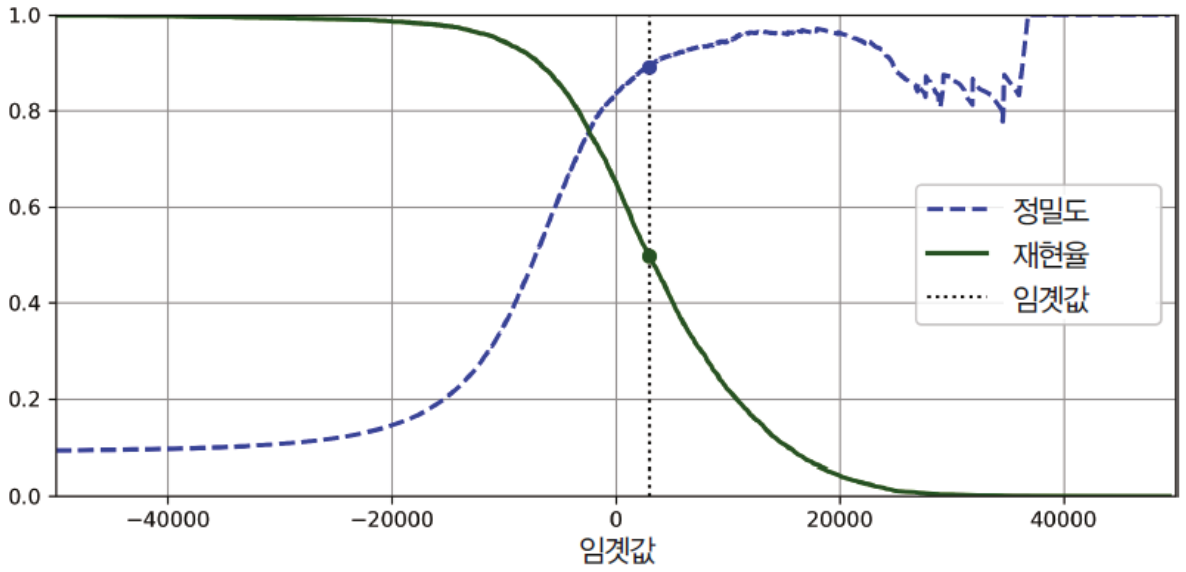

그림 3-5 결정 임곗값에 대한 정밀도와 재현율

3.3 성능 측정(12)

- 재현율에 대한 정밀도 곡선을 그려서 좋은 정밀도/재현율 트레이드오프를 선택

```
plt.plot(recalls, precisions, linewidth=2, label="정밀도/재현율 곡선")
[...] # 그리드, 범례, 레이블, 화살표, 텍스트를 추가합니다.
plt.show()
```

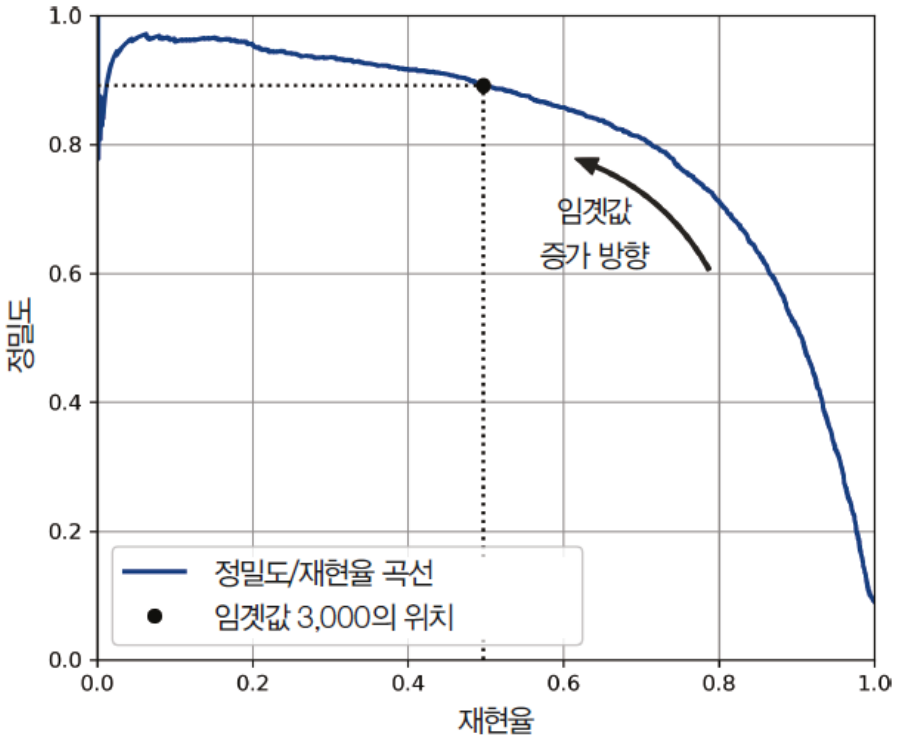

그림 3-6 정밀도와 재현율

3.3 성능 측정(13)

- 넘파이 배열의 argmax() 메서드
  - 정밀도가 최소 90%가 되는 가장 낮은 임곗값을 찾기

```
>>> idx_for_90_precision = (precisions >= 0.90).argmax()
>>> threshold_for_90_precision = thresholds[idx_for_90_precision]
>>> threshold_for_90_precision
3370.0194991439557
```

- (훈련 세트에 대한) 예측을 만들려면 분류기의 predict() 메서드를 호출하는 대신 다음 코드를 실행

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

- 예측에 대한 정밀도와 재현율을 확인

```
>>> precision_score(y_train_5, y_train_pred_90)
0.9000345901072293
>>> recall_at_90_precision = recall_score(y_train_5, y_train_pred_90)
>>> recall_at_90_precision
0.4799852425751706
```

3.3 성능 측정(14)

3.3.5 ROC 곡선

- 수신기 조작 특성(receiver operating characteristic, ROC) 곡선도
  - 거짓 양성 비율(false positive rate, FPR)에 대한 진짜 양성 비율(true positive rate, TPR, 재현율의 다른 이름)의 곡선
  - FPR 또는 폴-아웃(fall-out)은 양성으로 잘못 분류된 음성 샘플의 비율
    - 1에서 음성으로 정확하게 분류한 음성 샘플의 비율인 진짜 음성 비율(true negative rate, TNR)을 뺀 값
    - TNR – 특이도(specificity)라고도 함
  - ROC 곡선은 민감도(재현율)에 대한 1-특이도 그래프
  - roc_curve() 함수를 사용해 여러 임곗값에서 TPR과 FPR을 계산

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

3.3 성능 측정(15)

- 맷플롯립을 사용해 TPR에 대한 FPR 곡선 구현

```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]

plt.plot(fpr, tpr, linewidth=2, label="ROC 곡선")
plt.plot([0, 1], [0, 1], 'k:', label="랜덤 분류기의 ROC 곡선")
plt.plot([fpr_90], [tpr_90], "ko", label="90% 정밀도에 대한 임곗값")
[...] # 레이블, 그리드, 범례, 화살표, 텍스트를 추가합니다.
plt.show()
```

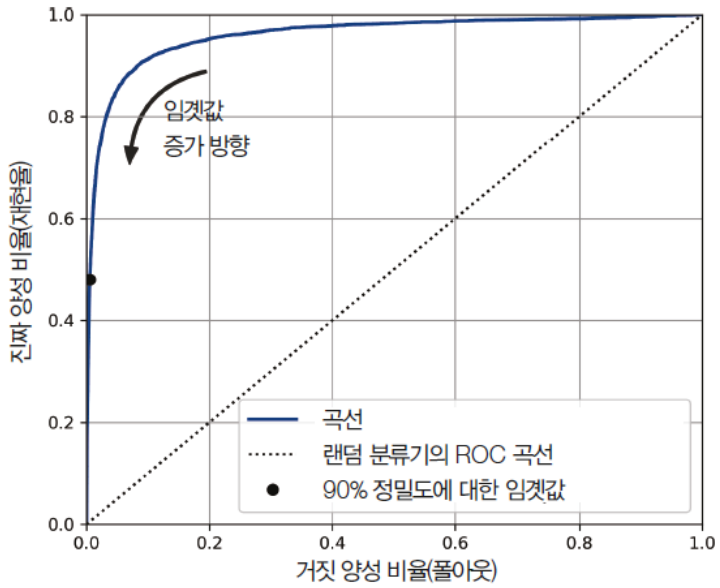

그림 3-7 가능한 모든 임곗값에서 진짜 양성 비율에 대한 거짓 양성 비율을 나타낸 ROC 곡선
검은 점이 선택한 비율의 지점(90%의 정밀도와 48%의 재현율)

3.3 성능 측정(16)

- 곡선 아래의 면적(area under the curve, AUC)을 측정해 분류기들을 비교
  - 완벽한 분류기는 ROC의 AUC가 1이고, 완전한 랜덤 분류기는 0.5

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
0.9604938554008616
```

  - RandomForestClassifier를 만들어 SGDClassifier의 PR 곡선과 F_1 점수를 비교

```
from sklearn.ensemble import RandomForestClassifier

forest_clf = RandomForestClassifier(random_state=42)
```

3.3 성능 측정(17)

- cross_val_predict() 함수
  - 교차 검증으로 RandomForestClassifier를 훈련하고 모든 이미지에 대한 클래스 확률을 예측

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                    method="predict_proba")
```

  - 훈련 세트에 있는 처음 두 개의 이미지에 대한 클래스 확률을 확인

```
>>> y_probas_forest[:2]
array([[0.11, 0.89],
       [0.99, 0.01]])
```

  - 두 번째 열에 양수 클래스에 대한 추정 확률이 포함되어 있으므로 이를 precision_recall_curve() 함수에 전달

```
y_scores_forest = y_probas_forest[:, 1]
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve(
    y_train_5, y_scores_forest)
```

3.3 성능 측정(18)

- PR 곡선을 그려서 서로 비교

```
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2,
         label="랜덤 포레스트")
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD")
[...] # 레이블, 그리드, 범례를 추가합니다.
plt.show()
```

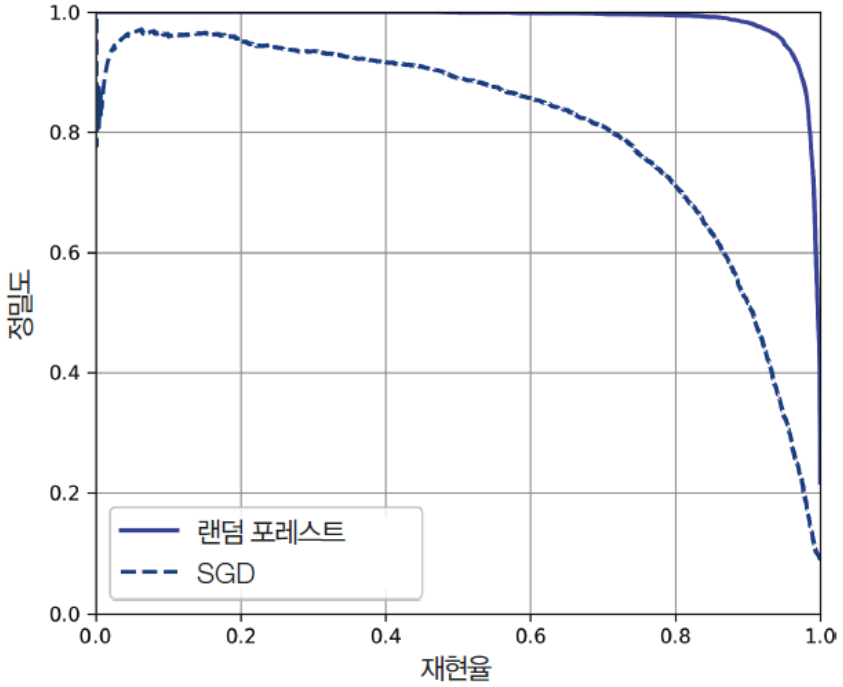

그림 3-8 PR 곡선 비교: 랜덤 포레스트 분류기가 SGD 분류기보다 훨씬 좋음
랜덤 포레스트의 PR 곡선이 오른쪽 위 모서리에 훨씬 가까워 AUC가 더 높기 때문임

3.3 성능 측정(19)

- [그림 3-8] RandomForestClassifier의 PR 곡선이 SGDClassifier의 곡선보다 훨씬 더 좋아 보임
  - 즉, 오른쪽 위 모서리에 훨씬 더 가까움
- F_1 점수와 ROC AUC 점수도 훨씬 더 우수

```
>>> y_train_pred_forest = y_probas_forest[:, 1] >= 0.5 # 양성 클래스 확률 ≥ 50%
>>> f1_score(y_train_5, y_train_pred_forest)
0.9242275142688446
>>> roc_auc_score(y_train_5, y_scores_forest)
0.9983436731328145
```

3.4 다중 분류(1)

- 다중 분류기(multiclass classifier) 또는 다항 분류기(multinomial classifier)는 둘 이상의 클래스를 구별
- OvR(one-versus-the-rest) 전략 또는 OvA(one-versus-all)
  - 이미지를 분류할 때 각 분류기의 결정 점수 중에서 가장 높은 것을 클래스로 선택
  - 대부분의 이진 분류 알고리즘에서는 OvR을 선호
- OvO(one-versus-one) 전략
  - 0과 1 구별, 0과 2 구별, 1과 2 구별 등과 같이 각 숫자의 조합마다 이진 분류기를 훈련
  - 장점 - 각 분류기의 훈련에 전체 훈련 세트 중 구별할 두 클래스에 해당하는 샘플만 있으면 됨
  - 작은 훈련 세트에서 많은 분류기를 훈련시키는 쪽이 빠르므로 OvO를 선호
- 다중 클래스 분류 작업에 이진 분류 알고리즘을 선택하면 사이킷런이 알고리즘에 따라 자동으로 OvR 또는 OvO를 실행

3.4 다중 분류(2)

- sklearn.svm.SVC 클래스를 사용해 서포트 벡터 머신 분류기를 테스트
  - 처음 2,000개의 이미지만 사용해 훈련

```
from sklearn.svm import SVC

svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000]) # y_train_5가 아닌 y_train을 사용합니다.
```

- 이미지에 대한 예측 만들기

```
>>> svm_clf.predict([some_digit])
array(['5'], dtype=object)
```

- decision_function() 메서드를 호출하면 샘플마다 총 10개의 점수(클래스마다 하나씩)를 반환

```
>>> some_digit_scores = svm_clf.decision_function([some_digit])
>>> some_digit_scores.round(2)
array([[ 3.79, 0.73, 6.06, 8.3 , -0.29, 9.3 , 1.75, 2.77, 7.21, 4.82]])
```

- 가장 높은 점수는 9.3이고 이는 클래스 5에 해당

```
>>> class_id = some_digit_scores.argmax()
>>> class_id
5
```

3.4 다중 분류(3)

- 분류기가 훈련될 때 classes_ 속성에 타깃 클래스의 리스트를 값으로 정렬하여 저장
  - 클래스 레이블 확인

```
>>> svm_clf.classes_
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object)
>>> svm_clf.classes_[class_id]
'5'
```

- OvO나 OvR을 사용하도록 강제하려면 OneVsOneClassifier나 OneVsRestClassifier를 사용
  - SVC 기반으로 OvR 전략을 사용하는 다중 분류기

```
from sklearn.multiclass import OneVsRestClassifier

ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

  - 예측을 만들고 훈련된 분류기 개수를 확인

```
>>> ovr_clf.predict([some_digit])
array(['5'], dtype='<U1')
>>> len(ovr_clf.estimators_)
10
```

3.4 다중 분류(4)

- 다중 분류 데이터셋에서 SGDClassifier를 훈련하고 예측

```
>>> sgd_clf = SGDClassifier(random_state=42)
>>> sgd_clf.fit(X_train, y_train)
>>> sgd_clf.predict([some_digit])
array(['3'], dtype='<U1')
```

- SGD 분류기가 각 클래스에 부여한 점수를 확인

```
>>> sgd_clf.decision_function([some_digit]).round()
array([[-31893., -34420., -9531., 1824., -22320., -1386., -26189.,
        -16148., -4604., -12051.]])
```

- cross_val_score() 함수를 사용해 이 모델을 평가

```
>>> cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy")
array([0.87365, 0.85835, 0.8689 ])
```

- 입력의 스케일을 조정하여 정확도를 89.1% 이상으로 높임

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler()
>>> X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
>>> cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3, scoring="accuracy")
array([0.8983, 0.891 , 0.9018])
```

3.5 오류 분석(1)

- 모델의 성능 향상을 위한 방법 - 생성된 오류의 종류를 분석
- 오차 행렬을 관찰
  - cross_val_predict() 함수를 사용해 예측을 만들고 confusion_matrix() 함수를 호출
- 오차 행렬을 컬러 그래프로 시각화 분석
  - ConfusionMatrixDisplay.from_predictions() 함수를 사용

```
from sklearn.metrics import ConfusionMatrixDisplay

y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```

- 오차 행렬을 정규화
  - normalize="true"로 지정하면 이 작업을 간단히 수행
  - values_format=".0%" 매개변수를 지정하여 소수점 없이 백분율을 표시

```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                        normalize="true", values_format=".0%")
plt.show()
```

3.5 오류 분석(2)

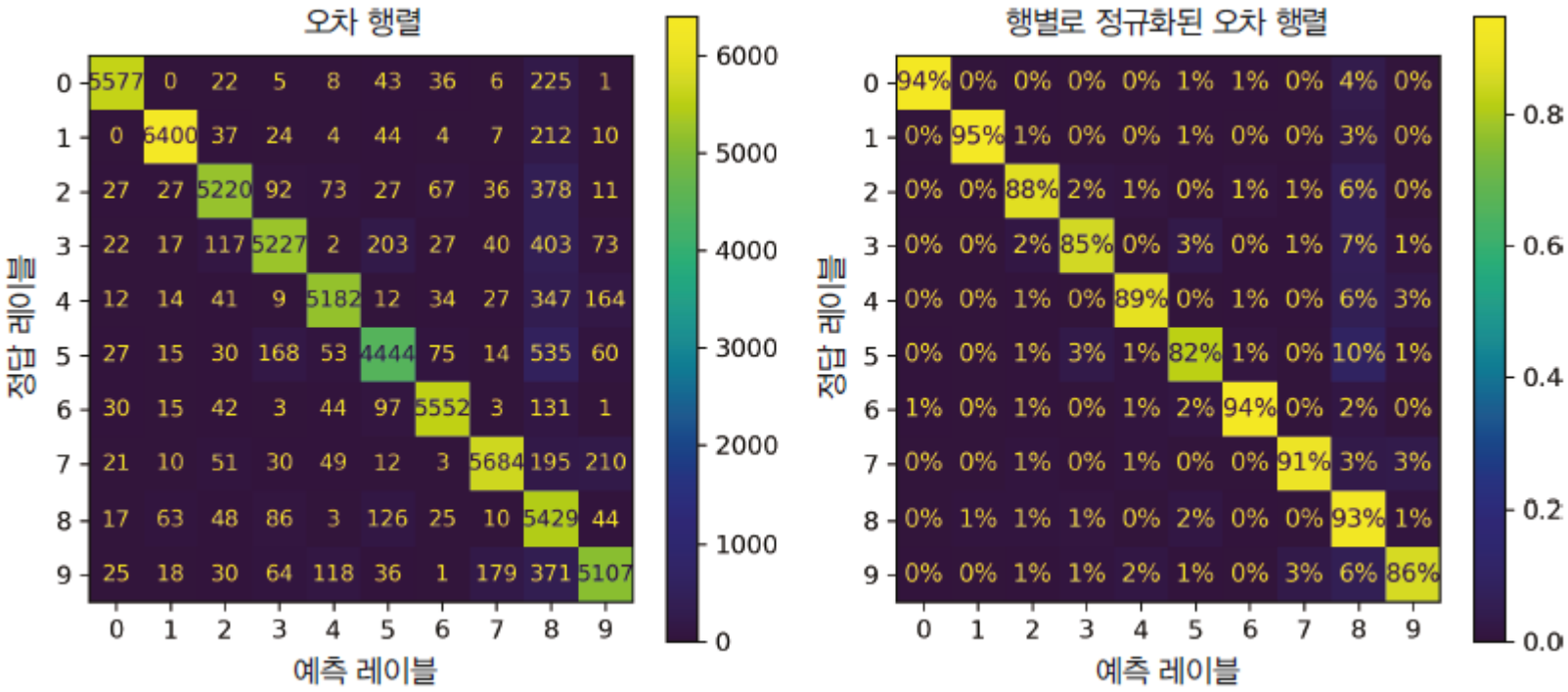

그림 3-9 오차 행렬(왼쪽)과 행별로 정규화된 오차 행렬(오른쪽)

3.5 오류 분석(3)

- 오류를 더 눈에 띄게 만들기 위해 올바른 예측에 대한 가중치를 0으로 설정
  - 오차 행렬을 행 단위가 아닌 열 단위로 정규화하려면 normalize="pred"로 지정

```
sample_weight = (y_train_pred != y_train)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                        sample_weight=sample_weight,
                                        normalize="true", values_format=".0%")
plt.show()
```

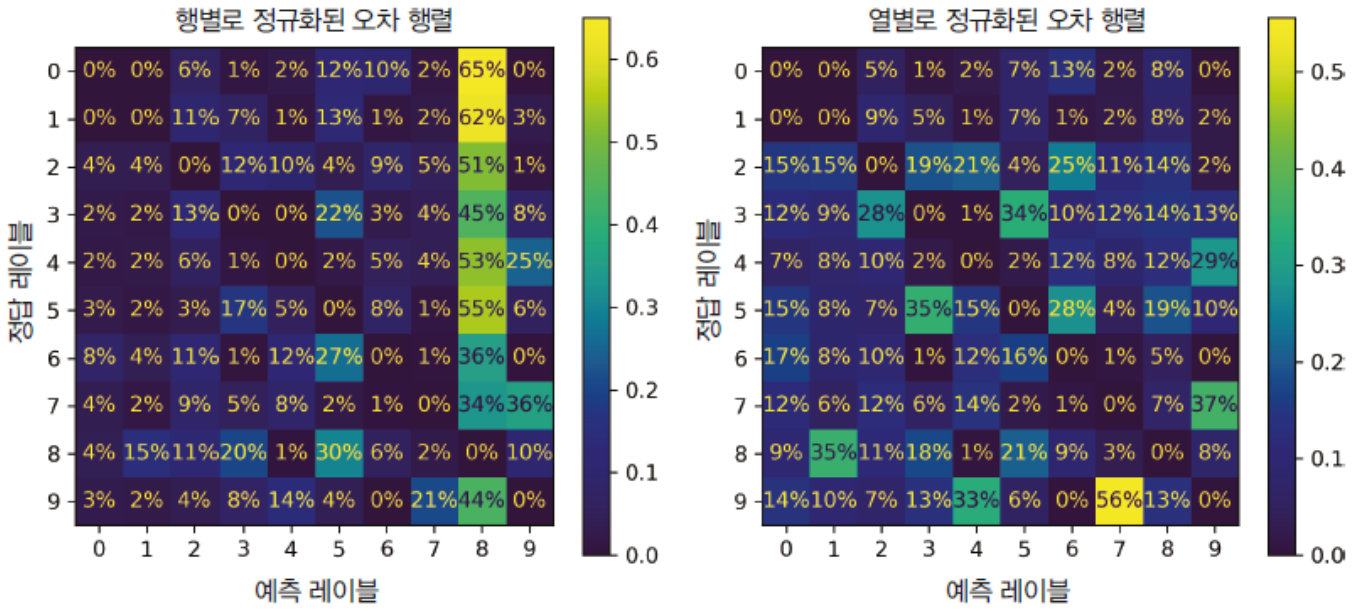

그림 3-10 행별로 정규화된 오차 행렬(왼쪽)과 열별로 정규화된 오차 행렬(오른쪽

3.5 오류 분석(4)

- 오차 행렬을 분석하면 분류기의 성능 향상 방안에 관한 인사이트를 얻음
  - 오차 행렬 스타일로 그린 3과 5의 샘플

```
cl_a, cl_b = '3', '5'
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]
[...] # X_aa, X_ab, X_ba, X_bb에 있는 모든 이미지를 오차 행렬 스타일로 그립니다.
```

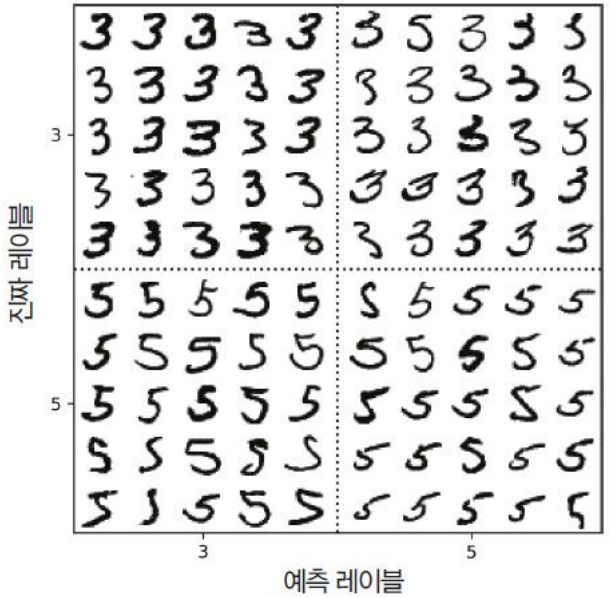

그림 3-11 오차 행렬 스타일로 그린 3과 5의 일부 이미지

3.5 오류 분석(5)

- 선형 분류기는 클래스마다 픽셀에 가중치를 할당하고 새로운 이미지에 대해 단순히 픽셀 강도의 가중치 합을 클래스의 점수로 계산
  - 몇 개의 픽셀만 다른 3과 5를 모델이 쉽게 혼동
- 3과 5의 주요 차이는 위쪽 선과 아래쪽 호를 이어주는 작은 직선의 위치
  - 숫자 3을 쓸 때 연결 부위가 조금 왼쪽으로 치우지면 분류기가 5로 분류하고 그 반대도 마찬가지
  - 분류기는 이미지의 위치나 회전 방향에 매우 민감
  - 3과 5의 오류를 줄이는 한 가지 방법은 이미지를 중앙에 위치시키고 회전되어 있지 않도록 전처리
- 데이터 증식(data augmentation)
  - 훈련 이미지를 약간 이동시키거나 회전된 변형 이미지로 훈련 집합을 보강

3.6 다중 레이블 분류(1)

- 다중 레이블 분류(multilabel classification) 시스템
  - 여러 개의 이진 꼬리표를 출력하는 분류 시스템
  - 각 숫자 이미지에 두 개의 타깃 레이블이 담긴 y_multilabel 배열 만들기
    - 첫 번째는 숫자가 큰 값(7, 8, 9)인지 나타내고 두 번째는 홀수인지 나타냄
    - 그다음 코드는 KNeighborsClassifier 인스턴스를 만들고 다중 타깃 배열을 사용하여 훈련

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

  - 예측을 만들면 레이블이 두 개 출력

```
>>> knn_clf.predict([some_digit])
array([[False,  True]])
```

3.6 다중 레이블 분류(2)

- 다중 레이블 분류기 평가
  - 각 레이블의 F_1 점수를 구하고(또는 앞서 언급한 어떤 이진 분류 지표를 사용하여) 간단하게 평균 점수를 계산
  - 이 코드는 모든 레이블의 가중치가 같다고 가정

```
>>> y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
>>> f1_score(y_multilabel, y_train_knn_pred, average="macro")
0.976410265560605
```

  - 레이블에 클래스의 지지도(support, 타깃 레이블에 속한 샘플 수)를 가중치로 주기
    - f1_score() 함수를 호출할 때 average="weighted"로 설정

3.6 다중 레이블 분류(3)

- SVC와 같이 기본적으로 다중 레이블 분류를 지원하지 않는 분류기를 사용하는 경우
  - 레이블당 하나의 모델을 학습
  - 이 전략은 레이블 간의 의존성을 포착하기 어렵게 할 수 있음
    - 문제를 해결하기 위해 모델을 체인(chain)으로 구성
  - 한 모델이 예측을 할 때 입력 특성과 체인 앞에 있는 모델의 모든 예측을 사용
    - ClassifierChain 클래스는 훈련에 진짜 레이블을 사용하며 체인 내 위치에 따라 각 모델에 적절한 레이블을 공급

3.6 다중 레이블 분류(4)

- 교차 검증 전략을 사용하여 ClassifierChain을 만들고 훈련하는 방법
  - 속도를 높이기 위해 훈련 세트에서 처음 2,000개 이미지만 사용

```
from sklearn.multioutput import ClassifierChain

chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

  - ClassifierChain을 사용해 예측

```
>>> chain_clf.predict([some_digit])
array([[0., 1.]])
```

3.7 다중 출력 분류(1)

- 다중 출력 다중 클래스 분류(multioutput-multiclass classification), 다중 출력 분류(multioutput classification)
  - 다중 레이블 분류에서 한 레이블이 다중 클래스가 될 수 있도록 일반화(즉, 값을 두 개 이상 가질 수 있음)
- (학습 예제) 이미지에서 잡음을 제거하는 시스템
  - 잡음이 많은 숫자 이미지를 입력으로 받아, 깨끗한 숫자 이미지를 MNIST 이미지처럼 픽셀의 강도를 담은 배열로 출력
  - MNIST 이미지에서 추출한 훈련 세트와 테스트 세트에 넘파이의 randint() 함수를 사용하여 픽셀 강도에 잡음을 추가

```
np.random.seed(42) # 동일하게 재현되도록 하기 위해 지정합니다.
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise
y_train_mod = X_train
y_test_mod = X_test
```

그림 3-12 잡음이 섞인 이미지(왼쪽)와 깨끗한 타깃 이미지(오른쪽)

3.7 다중 출력 분류(2)

- 분류기를 훈련시켜 이미지를 깨끗하게 만들기

```
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
plot_digit(clean_digit)
plt.show()
```

그림 3-13 깨끗하게 정리된 이미지

연습문제

1. MNIST 데이터셋으로 분류기를 만들어 테스트 세트에서 97% 정확도를 달성하기
2. MNIST 이미지를 (왼, 오른, 위, 아래) 어느 방향으로든 한 픽셀 이동시킬 수 있는 함수 만들기
   - 훈련 세트에 있는 각 이미지에 대해 네 개의 이동된 복사본(방향마다 한 개씩)을 만들어 훈련 세트에 추가
   - 확장된 데이터셋에서 앞에서 찾은 최선의 모델을 훈련시키고 테스트 세트에서 정확도를 측정 모델 성능이 더 높아졌는지 확인
   - 데이터 증식 또는 훈련 세트 확장(training set expansion)
3. 타이타닉 데이터셋에 도전해보기
   - https://www.kaggle.com/c/titanic 또는 https://homl.info/titanic.tgz에서 데이터를 다운로드
   - 다른 열을 기반으로 Survived 열을 예측할 수 있는 분류기를 훈련
4. 스팸 분류기를 만들어보기

연습문제 정답은 https://github.com/rickiepark/handson-ml3에 있는 주피터 노트북에서 확인